2026 · Technical knowledge share

Software Development in the AI Era

Code Test Secure Deploy



AI speeds implementation but responsibility stays with me

How AI Is Changing Development

- AI drafts code, tests, docs and boilerplate
- Cycles get faster
- Output can hide bugs and wrong assumptions
- Focus shifts to verification and judgment

Traditional: Spec

Write

Review

Test

Ship



AI speeds implementation but responsibility stays with me

The New Developer Mindset

- Write code → verify code
- Generate features → deliver reliable software
- Fix symptoms → find root causes
- Trust AI output → validate it

AI generates

Human review

Automated validation

Production monitoring



AI speeds implementation but responsibility stays with me

Testing Fundamentals

- Unit: one function
- Component: render and behavior
- Integration: pieces working together
- E2E: full user journey
- Regression: old bugs stay fixed
- API/contract: client and server agree





Testing a Login Feature

- Unit: validateEmail()
- Component: form renders, button disables
- Integration: validation + API + UI update
- Browser/E2E: sign in in a real browser

Unit: validateEmail

Component: LoginForm

Integration: form + API

E2E: full sign-in flow



AI Code Review and Auditing

- Hunt logic errors and hidden bugs
- Spot duplicate or needlessly complex code
- Check edge cases and error handling
- Confirm architectural consistency
- Never accept AI changes blindly

Open PR

Automated review

Human review

Fix issues

Merge



Static Analysis and Type Safety

- TypeScript strict mode
- Linting and formatting checks
- Catch unused code and unsafe types
- Build-time validation

> Type error: any not allowed

> Lint warning: unused var

> Format: Prettier OK

> Build: passed



Application Security

- XSS and CSRF
- Broken access control
- Secret and credential leaks
- Vulnerable dependencies
- Validate every input
- Hiding UI is not authorization: enforce on the server

Client

Input validation

Server-side authz

Protected API



Performance Engineering

- Core Web Vitals: LCP, INP, CLS
- Bundle size, lazy loading, caching
- React rendering performance
- Profile, then real-user monitoring

LCP: loading

INP: responsiveness

CLS: layout stability

Profile bottlenecks



Accessibility and Inclusive UI

- Semantic HTML and labels
- Keyboard navigation and focus
- Screen readers; ARIA only when needed
- Color contrast and accessible dialogs
- Automated plus manual testing

Keyboard pass

Screen reader pass

Contrast check

Automated scan



API Integration and Contracts

- Validate requests and responses
- Handle status codes and errors
- Loading, empty, success, failure states
- Timeouts, retries, pagination
- Keep frontend and backend in sync

UI

Zod validation

API (OpenAPI contract)

Error path: retry / message



Error Handling and Observability

- Runtime error tracking
- Structured logs
- Root-cause analysis
- Watch frontend and API failures
- Find problems after deploy

Error raised

Alert

Stack trace

Root cause

Fix and release



CI/CD and Quality Gates

- Lint, types, tests on every PR
- Automated builds and dependency checks
- Block deploys on critical failures
- Preview deployments
- Rollback plan

Commit

Tests + lint

Security checks

Deploy

Monitor



AI Assistants and Agents

- Clear prompts with requirements
- Give relevant codebase context
- Review diffs, check assumptions
- Limit permissions; protect secrets
- Verify generated tests independently

Prompt

Agent proposes diff

Human approval

Tests run



Git, Refactoring, Regression Prevention

- Small, focused commits
- Review diffs before merge
- Refactor with tests in place
- Spot risky changes; plan rollbacks

Small commit

Pull request

Regression tests pass

Merge



Architecture and Maintainability

- Clear component boundaries
- Reusable logic, sensible state
- Avoid needless abstraction
- Manage technical debt
- Microfrontends only when justified

Modular: UI

Hooks

Services

API client



Cross-Browser and Responsive QA

- Chrome, Firefox, Safari
- Mobile, tablet, desktop
- Hydration and layout regressions
- Emulators vs real devices

Desktop

Tablet

Mobile

Real device check



Requirements and Acceptance Criteria

- Turn stories into testable rules
- Define behavior before building
- List edge cases and business rules
- Check against design and API specs
- Stop AI solving the wrong problem

User story

Acceptance criteria

Implementation

Tests

Approval



Risk-Based QA and Release Readiness

- Rank bugs by severity and impact
- Test critical journeys first
- Scrutinize payment, auth, permissions
- Keep test evidence
- 100% coverage does not mean bug-free

High impact + likely: fix now

High impact, rare: guard

Low impact: schedule

Checklist: evidence kept



A Complete AI-Era Workflow

- Requirements define the target
- AI implements; humans review
- Static analysis and tests gate changes
- Security and browser QA verify
- CI/CD ships; monitoring feeds back

Requirements

AI implementation

Code review

Static analysis

Automated tests

Security review

Browser QA

CI/CD

Monitoring



Auditing an AI-Built Employee CRUD

- Employee data and validation correct?
- Authentication and role-based access
- API error handling
- Unit, integration, browser tests
- Responsive UI and accessibility
- Performance and monitoring

Validation

Roles

Errors

Tests

Responsive + a11y

Monitoring



A Practical Learning Roadmap

- L1 Essential: testing, TypeScript, ESLint, review, API and security basics
- L2 Production: CI/CD, performance, accessibility, regressions, monitoring
- L3 Advanced: architecture audits, AI agents and MCP, risk-based QA, release engineering

Level 1: Essential

Level 2: Production readiness

Level 3: Advanced



Key Takeaways

- AI code is not automatically production-ready
- Tests are essential, never complete
- Security, performance, accessibility, maintainability matter
- Verify requirements, behavior, risk
- Goal: reliable software, not more code

AI

Engineering judgment

Automated verification

Production quality



AI speeds implementation but responsibility stays with me

References and Further Learning

- nextjs.org/docs/app/guides/testing
- playwright.dev/docs/intro
- vitest.dev/guide/
- testing-library.com/docs/react-testing-library/intro/
- owasp.org
- web.dev/articles/vitals
- docs.github.com/en/actions

Official docs

Verify tool details

Keep learning

